# 모바일 아두이노 임베디드 플랫폼 설계*

이 아 리** · 홍 선 학**

# *Mobile Arduino Embedded Platform Design*

Lee, Ah Ri · Hong, Sun Hag

〈Abstract〉

In this paper, we implemented the pattern matching with the Arduino and App Inventor platform under the bluetooth mobile environment between Android phone and Arduino Platform. The combination between Arduino and App Inventor makes the feasibility of Android programming easy by wireless communications and provides the opportunity to broaden the functionality for mobile device. We used the softwares which were Arduino IDE, VC++, OpenCV, Processing and App Inventor. And also compared the performance of mobile Arduino platform with LabView GUI programming, we reduced the usage of libraries that compiled and executed the pattern matching programming. We experimented the mobile embedded platform performance under bluetooth communication and verified the functionality of the mobile Arduino platform design for identifying the pattern matching.

Key Words : Arduino, App Inventor, Embedded, Pattern Matching

## Ⅰ. Introduction

The need for mobile programming based Android devices is increasing because the number of Android devices have been incremented and the usefulness of them enhance the accessibility to information. And also, Android app programming is indispensable to the mobile devices with its extendable and flexibility from which many kinds of research such as communication, game, and embedded fields.

The main difficulties in Android programming come from combining the Java and Eclipse program skill and SDK structure. It is obvious that users like us need a graphical programming for easily building the entry level Android development environment.

And also instead of Java programming, it can be easily implemented by App Inventor which was first announced as a GUI and online programming environment with Google Lab in late 2010, and had transferred to MIT mobile Learning center for public use under the sprit of open source.

* 본 연구는 2012학년도 서일대학 교내학술연구비지원으로 수행하였음

** 서일대학교 컴퓨터전자과 교수



www.kci.go.kr

At the same time, we have used the Uno which used the AVR(ATmega328) since 2010. It has a smaller bootloader called OptiBoot that runs at 0.5KB and a flash memory of 32KB. All of the Arduinos have used an FTDI chip to convert the TTL serial from Atmel ATmega. This allows for printable debugging, connecting to software like PureData/Max, Processing, Python, etc. It also allows us to update firmware via serial bootloader.

We found out that it is not a problem what kinds of tool about making the mobile embedded platform, but how to easily build the Android programming from combining software(App Inventor and Arduino IDE) and hardware (Arduino and sensors) in an entry level of mobile programming environment. In this paper we would provide the mobile Arduino embedded platform with Android system through App Inventor[1-3].

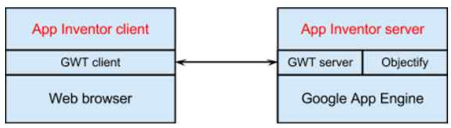
## Ⅱ. Basic Theory

This chapter surveys a fundamental overview of understanding the App Inventor which has two main components, one of which is Designer that is a webpage where we selects the components for our app and adjust attributes of each components, the other is Blocks Editor that is a java window where we put the program blocks together to decide how the components should behave.

### 2.1 App Inventor structures

Google Web Toolkit (GWT) allows programmers to write client-server applications in Java without worrying about the details of remote procedure calls (RPCs). GWT compiles the client code into JavaScript, which runs within a web browser, and the RPCs run as Java code on the server, with communication done via HTTP.

Google App Engine (GAE) is a cloud-computing platform that enables programs written in Java (or Python) to run and maintain data on Google servers. The original internal version of App Inventor was built directly on proprietary Google infrastructure but, for the open source release, was rewritten to use GAE, using the third-party Objectify datastore API. GWT and GAE play well together, where a GWT server can run on a GAE server. <Figure 1> shows the App Inventor works.



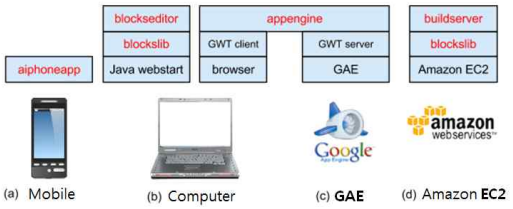

<Fig 1> App Inventor Structure

<Figure 2> shows where each of the projects' code runs. It is simplified in that it only shows the name of the project and not specific build targets. Greater detail can be found in documents on each project and in the Ant build.xml files in each project's directory. Also note that although the diagrams depict Amazon EC2 as a potential platform to deploy part of the system, its use is not mandatory.

The App Inventor consists of the following subdirectories, the first seven of which contain source code for sub-projects: aiphoneapp is the interpreter that runs on the mobile or emulator, appengine is the GWT application that provides the

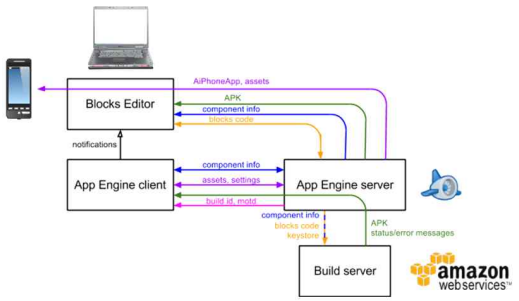


www.kci.go.kr




<Fig 2> App Inventor Components

Designer JavaScript code to the client browser. blockseditor is the Blocks Editor which runs through JNLP on the client, blockslib is the library code used by both blockseditor and buildserver, buildserver is an http server/servlet that takes a source zip files as input and produces an apk and/or error messages. <Figure 3> represents the runtime communication among sub-projects, and shows dynamic interactions among the App Inventor sub-projects. When the browser client starts, a variety of information, is passed from the App Engine server to the client. This includes the server build id and the message of the day. Information about a project's components, assets, and user settings flows in both directions.




<Fig 3> Runtime Communication

If the user establishes a connection from the Blocks Editor to the device or emulator, the Blocks Editor requests and receives AiPhoneApp and the current project's assets, which it passes through to the device.

## 2.2 App Inventor Components

One aspect of Android worth highlighting and reading about is the Android UI thread, which is responsible for responding to UI interactions and updating the UI. All user code and most component library code runs in the UI thread. This gives a simple execution model for us. An App Inventor procedure will never be interrupted by an event handler, or vice versa, nor will one event handler be interrupted by another.

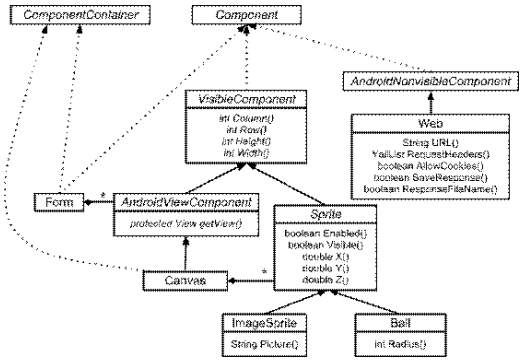
All components fit within a class/interface hierarchy, a subset of which is shown in <Figure 4>. All components implement the Component interface, which consists mostly of useful constants. Every component extends either the VisibleComponent or AndroidNonvisible Component abstract classes, and some also implement the ComponentContainer interface. Every application has a Form, which holds zero or more instances of AndroidViewComponent, the superclass of most visible components. Exceptions include Sprite, which can only be contained in a Canvas.

## 2.3 GUI Elements

The major panels are TopPanel, DeckPanel, StatusPanel, and RootPanel which are provided by GWT and fills the application window. The remaining panels are created in Ode.initializeUi().



www.kci.go.kr




<Fig 4> Component class/interface Hierarchy

The first to be created is mainPanel, which fills the RootPanel. The Design tab is where the user adds components to a project, specifies their properties and positions, and manages media files.

The ViewerBox contains a ProjectEditor. The EditorManager maintains multiple ProjectEditor instances, one for each project opened in the current browser session. In turn, each ProjectEditor has a FileEditor for every screen in the project.

The Projects tab, is where a user creates, deletes, opens, uploads, or downloads projects and keystores. The ProjectToolbar, contains four instances of ToolbarItem, each of which has a name, a caption, and a Command. The ProjectListBox is populated by ProjectList, which contains a table with “Name” and “Date Created” editors that can be used for sorting, and rows consisting of a CheckBox, a project name, and a creation date. The Debugging tab is offered only if AppInventorFeatures. hasDebuggingView() returns true. It consists of a WorkAreaPanel that contains two boxes[4-5].

## Ⅲ. Development Environment

In this paper, we used the Kawa is a free implementation of Scheme that compiles to Java byte code, which can be converted by the tool dx into Dalvik bytecode. We used Scheme as the internal representation of users’ programs, and part of the runtime library is written in Scheme. These get compiled down to byte code and linked with our components library written in Java, and external libraries written in Java and C/C++, by the buildserver.

### 3.1 Component Information files

Several parts of the system need to know what components exist and their characteristics. This information is generated at build time through a set of annotation processors that run over the components source code and generate files required in order to build the other App Inventor projects. These files can be found in the subdirectory appinventor/build /componentsSerial Libraries. The simple_components. json file is used by the editor within the Designer in the GWT client.

The simple_components_permissions.json file is used by the buildserver to generate the permissions required by an app, has entries with the names and required permissions of each component. The ya_lang_def.xml file contains the specification of the blocks used by blockseditor, including both static library functions and components with their methods, properties, and events. Its header and footer come from the static files output_header.txt and output_ footer.txt in the "src/com/google/app



www.kci.go.kr

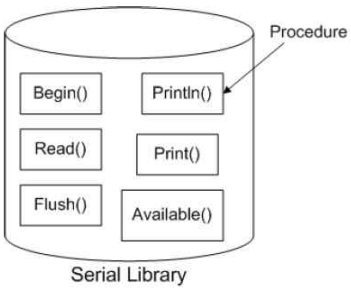
inventor /components/scripts/templates" directory within the components project[6-8].

### 3.2 Arduino Platform Bridge

We experimented pattern matching with Arduino for examining the performance of embedded platform which includes Bluetooth communication via OpenCV and Processing 2.0.

Software library is a big collection of procedures, where all the procedures are related. This library allows the Arduino to send data back to the computer. One of the best features of the Arduino project is the ability to add on pre-crafted libraries that add hardware support. <Figure 5> represents the structure of Serial library.

There's tons of them, and we could pick and choose which to install. They're only loaded in when the sketch we're working on needs.




<Fig 5> Serial Library Structure

As before we said, a library is a folder with some files in it, the files will end in ".cpp" and ".h". There may also be some .o files[9-10].

### 3.3 Android Communication

In this paper, we used the DFrobot Bluetooth Bee wireless module. It has compact size and its pinout is compatible with XBEE which is suitable for all kinds of micro controller systems who have 3.3V power out. The Bluetooth Bee modules comes with an on-board antenna which provides better signal quality. It acts like a transparent serial port, which works with a variety of Bluetooth adapter and Bluetooth phone. Bluetooth Bee wireless module can modify the baud rate to use XBEE adapter. If necessary, we could use XBEE Arduino controller receives the base plug, enabling Bluetooth wireless control.

In this paper, we used Processing, for App development. Processing is a daughter project of Arduino. Processing is designed as an easy gateway into programming. As of version 2.0, it allows us to compile and run code directly on an Android device.

To use the Arduino ADK, we need to install a plug in for Processing called Arduino ADK. This plug in will make sure that the libraries and xml files are included in the compilation of our code, and send the program to our device. Our Processing sketches must also include a new library that enables communication between the device and accessory devices over the USB port[11-12].

## Ⅳ. Mobile Embedded platform Design

In this section, we described the characteristics of mobile embedded platform which let us create a



www.kci.go.kr

pattern matching app for android phones by piecing together programming blocks in a web browser.

### 4.1 Supported Platform architecture

Building and deploying an App Inventor server can be done using MacOS, GNU/Linux, or Windows operating systems. App Inventor is a Java service built on Google App Engine. As with any App Engine service, we would deploy our App Inventor service to Google's App Engine infrastructure at appspot.com, so that anyone on the Web can access it, provided we give them permission.

The App Inventor service uses an auxiliary Build Server that handles the computationally intensive work of compiling and packaging apps.

If we're running App Inventor on our local machine that can act as its own build server. If we deploy App Inventor service on at appspot.com, we could obtain and configure a machine to run as the build server. The build server machine must be a Mac OS or GNU/Linux machine that is accessible to the machine that hosts our App Inventor service.

### 4.2 Building and Launching App Inventor Components under GNU/Linux

Java development tool(JDT) allows us to write, compile, test, debug, and edit programs written in the Java programming language. Before starting with App Inventor, we should did the setup of the following software:

- Java: We have tested build using Sun Java 6. Other versions might work, but we haven't tested in this paper. We recommend using Sun Java as opposed to OpenJDK. In particular, when we used App Inventor to create apps, the Blocks Editor will not work under OpenJDK due to the fact that the needed utility to sign jar files "jarsigner" is not provided with OpenJDK. Once we've installed Java, check that Java and Java Web Start both work by trying the standard App Inventor "test our Java configuration" tests at the below site, http://beta.appinventor.mit.edu/learn/setup/index.html.
- Apache ant: Install Apache ant, or we may need to update our installed version to 1.8.1 or 1.8.2.
- bash: Run bash apache-ant-1.8.2
- Google AppEngine Java SDK: Get and install the latest Java version of the SDK[13-15].

### 4.3 Mobile embedded platform configuration

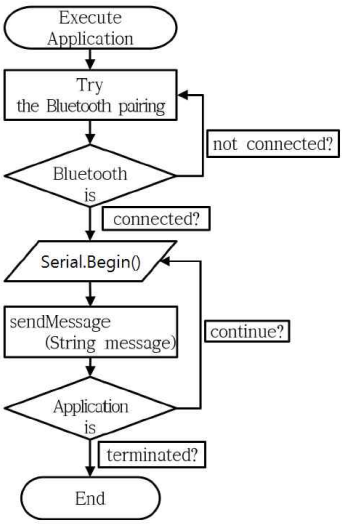
We configured the bluetooth and camera for gathering an image and communicating with devices. At first when an app is executed, bluetooth tried to pair between Android mobile and Arduino platform. If pairing is success, string message is sent to the mobile like <Figure 6>.

Serial.begin() function takes a single parameter which is a desired communication speed such as 9,600 baud, but make sure both sides use the same speed. The write(byte[] buffer) method sends text via bluetoothChatService.java().

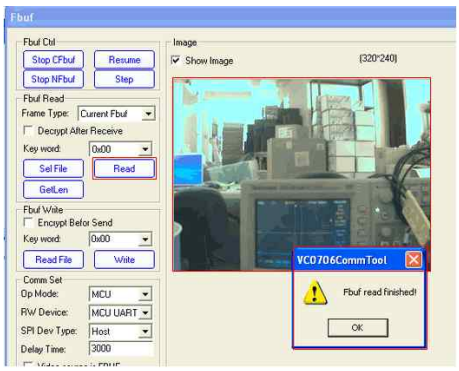
<Figure 7> represents the setup of Menu and Image in TTL camera which captured an image during experiments.[16-18].



www.kci.go.kr




<Fig 6> Bluetooth Procedure

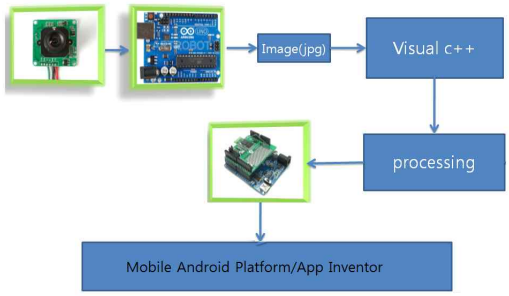



<Fig 7> Configuration of TTL Serial Camera

## 4.4 Experiments for Pattern Matching

We experimented the mobile embedded platform with Arduino to chord pattern matching with OpenCV and Processing in <Figure 8>.

In this paper, We used the TTL CMOS camera and Bluetooth module for hardware, and used software such as VC++, OpenCV, Processing and App Inventor as an appropriate tool in information technology and Android as listed below : 1. Lower




<Fig 8> System Configuration

the entry level down to beginners. 2. Easy to understand and less programming skills required. 3. More motivation and willing to challenge. 4. The easiest way to make Android programming with mobile so far.

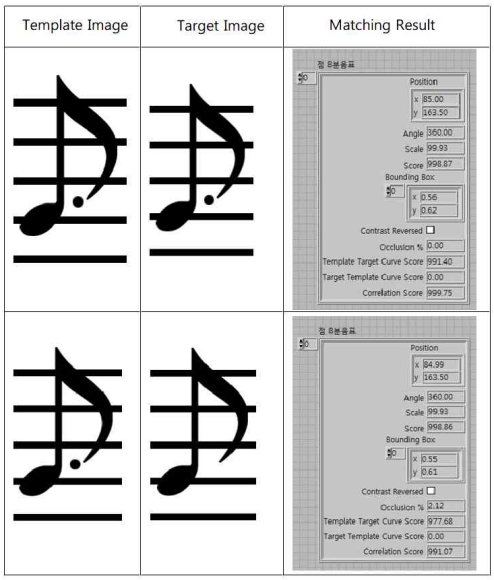
On the other hand, App inventor is some how limited that the codes are hard to manage when getting larger and it is not easy to implement a large scale complex application. Nonetheless we used App inventor as a middleware to practice and to build up our platform design.

<Figure 9> represents the pattern matching with GUI programming, LabVIEW compared to Arduino platform. Machine vision used the edge based geometric matching method for identifying between template image and target image in sacrificing a lot of the computation time and libraries.

Edge based geometric matching compared template image with target image using the scale means the relative size with percent, the score is complete matching value with max value being 1000, the occlusion means the cross section with each objects for percent and the correlation score means the correlation value for pixel strength with max value being 1000.

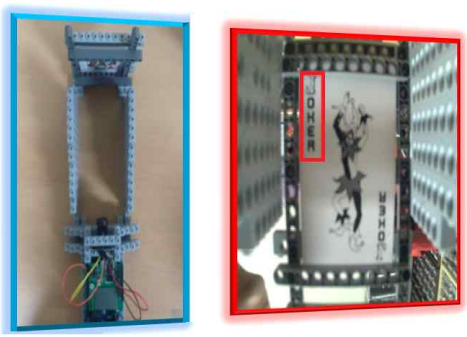


www.kci.go.kr




<Fig 9> Pattern matching with Machine Vision

<Figure 10> represents the pattern matching experiment result with Arduino Embedded platform. We experimented the mobile platform with App Inventor as far as we are desired level, afterwards we will keep track with the development of app inventor's components and mobilization in this filed[19-20].



<Fig 10> Pattern matching result

## Ⅴ. Conclusions

In this paper, we implemented the pattern matching with Arduino platform and Android environment. App Inventor helps the connection of mobile device by wireless communication and is a terminal for building access point for Android with Web browser. We demonstrated that the mobile Arduino embedded platform with App Inventor and enhance the functionality of pattern matching compared to Machine Vision system in respect of saving the computational time and of reducing the number of libraries. Afterwards we would develop the component basis tools of Android mobile devices for easily adapting the pattern matching in the pursuit of selecting the inferior goods in industrialized fields

## 참고문헌

[1] Ladyada, "Arduino Tips Tricks & Techniques," Adafruit learning system, 2013.

[2] 남상규, "임베디드 엔지니어를 위한 리눅스 커널," Aesop, 2009.

[3] 홍선학, 조도현, "모바일로 배우는 아두이노 따라하기," 성안당, 2012.

[4] David Wolber, "App Inventor: Create your own Android Apps," O'Reilly, 2011.

[5] Massachusetts Institute of Technology, "MIT App Inventor," 2012.

[6] Chi-Hung Tseng, Gi-Ming Liu, Wen-You Lu, "Combination Curriculum of Robotics and Mobile Phone," International Journal of Engineering Education, 2012.



www.kci.go.kr

[7] Jason Tyler, “App Inventor for Android: Build Your Own Apps,” Paperback, 2011.
[8] Ralph Roberts, “Google App Inventor,” Paperback, 2011.
[9] 정성환, 이문호, “OpenCV를 이용한 컴퓨터 비전 실무 프로그래밍,” 홍릉과학출판사, 2007.
[10] Casey Reas and Ben Fry, “Processing : A programming Handbook,” O’Relley, 2010.
[11] Donn Felker, “Android Application Development For Dummies,” Paperback, 2011.
[12] Reto Meyer, “Professional Android 2 Application Development,” Wrox, 2010.
[13] 노성동, 이명의, “임베디드 리눅스 구조와 응용,” GS인터비젼, 2009.
[14] Wolfgang Maurer, “Professional Linux Kernel architecture,” Wiley Pub, 2010.
[15] Robert Love, “Linux System Programming,” O’Reilly Media, 2007.
[16] 조나단 코벳, “리눅스 디바이스 드라이버,” 한빛미디어, 1999.
[17] 홍선학, “UML기반의 창의공학용 로봇설계,” 한국통신학회, 제33권, 제10호, 2008, pp. 343-349.
[18] 홍선학, “영상 추적을 이용한 이동로봇제어,” 대한전자공학회, 제42권, 제4호, 2005, pp. 201-208.
[19] 홍선학, 윤진섭, “임베디드 RTOS기반의 로봇 컨트롤러 설계,” 디지털 산업정보학회, 제6권 제 4호, 2010.
[20] 홍선학, 이아리, “모바일 임베디드USN 플랫폼 설계,” 한국통신학회, 제37권 제4호, 2011.

## ■ 저자소개 ■



이 아 리
Lee, Ah Ri

2009년 3월~현재
서일대학 컴퓨터전자과 강사
2003 2월 광운대학교 대학원(이학박사)

관심분야 : 이미지 매칭, 그래픽 알고리즘, 계산기학
E-mail : hidasom@chol.com



홍 선 학
Hong, Sun Hag

1992년 ~ 현재
서일대학 컴퓨터전자과 교수
1994년 광운대학교 대학원 박사 졸업.

관심분야 : 제어 응용, 리눅스 임베디드
E-mail : hongsh@seoil.ac.kr

논문접수일 : 2013년 11월 8일
수 정 일 : 2013년 11월 20일
게재확정일 : 2013년 11월 28일



www.kci.go.kr